

NetVis User Manual

Network Topology Visualization Engine

Simon Knight

Adelaide, Australia

March 2026

About this manual. This manual covers the full operational use of NetVis, a high-performance network topology visualization engine written in Rust. Topics include rendering topologies as SVG, PNG, and PDF; choosing and tuning layout algorithms; overlaying live traffic utilization data; visually diffing topology snapshots; temporal analysis across time-series captures; and interactive editing via the embedded WASM component. Prerequisite knowledge of basic YAML syntax and familiarity with command-line tooling is assumed.

Contents

1	Quick Start	1
2	Installation & Prerequisites	1
2.1	Installing the Native CLI	2
2.2	Python Bindings	2
2.3	WASM Component for Browser / Workbench	2
3	Core Workflows	2
3.1	Basic Rendering	2
3.2	Hierarchical Datacenter Visualization	3
3.3	Geographic WAN Maps	3
3.4	Traffic Overlay and Animation	4
3.5	Topology Diffing	5
3.6	Temporal Analysis	5
4	Layout Algorithm Guide	6
4.1	Algorithm Summary	7
4.2	Selecting an Algorithm	7
4.3	Composite Layouts	7
4.4	Auto-Tuner	8
5	YAML Topology Reference	8
5.1	Top-Level Structure	8
5.2	Node Fields	8
5.3	Edge Fields	9
5.4	Groups	9
5.5	Layers	9
5.6	Inline Traffic Data	9
5.7	Annotations	9
6	CLI Command Reference	10
6.1	Core Commands	10
6.2	Layout Flags	10
6.3	Appearance Flags	10
6.4	Filtering Flags	11
6.5	Traffic Overlay Flags	11
6.6	Diff and Analysis Flags	11
6.7	Advanced Flags	12
7	Themes and Level of Detail	12
7.1	Available Themes	12
7.2	Level of Detail (LoD)	12
7.3	Custom Themes	13
8	Troubleshooting	13
9	Integration with Other Tools	14
9.1	TopoGen — Topology Generation	14
9.2	netsim — Simulation Results Overlay	15
9.3	AutoNetKit — Topology Modelling	15
9.4	NetBox — IPAM/DCIM Adapter	15

9.5	Workbench — Embedded WASM Component	16
10	Further Reading	16
A	Topology YAML Schema	16
B	Configuration Reference	17
B.1	CLI Flags	17
B.1.1	Input and Output	17
B.1.2	Layout	17
B.1.3	Appearance	18
B.1.4	Canvas and Viewport	18
B.1.5	Force-Directed Tuning	18
B.1.6	Geographic Layout	18
B.1.7	Filtering	19
B.1.8	Analysis Features	19
B.1.9	Quality and Diagnostics	19
B.2	CLI Subcommands	20
B.3	JSON Configuration File	20

1 Quick Start

Get a topology diagram on screen in under three minutes.

Step 1. Create a minimal topology file `topology.yaml`:

```
# topology.yaml - minimal NetVis input
topology:
  name: quickstart
  nodes:
    - id: core1
      label: "Core-1"
      node_type: router
    - id: core2
      label: "Core-2"
      node_type: router
    - id: sw1
      label: "Access-SW1"
      node_type: switch
  edges:
    - from: core1
      to: core2
      label: "10G ISL"
    - from: core1
      to: sw1
    - from: core2
      to: sw1
```

Step 2. Render the topology to SVG:

```
netvis --input topology.yaml -o output.svg
```

Open `output.svg` in any modern browser to inspect the result.

Step 3. Try a different layout algorithm:

```
netvis --input topology.yaml --layout hierarchical
```

The default output file is `output.svg` when `-o` is omitted.

Step 4. Export to PNG at higher resolution:

```
netvis --input topology.yaml --format png --dpi 150 -o output.png
```

2 Installation & Prerequisites

▷ Prerequisites

- **Rust 1.93+** — required for building the native CLI binary from source (`cargo install netvis`). Pre-built binaries are available and have no Rust runtime dependency.
- **Python 3.8+** — required only if using the Python bindings (`pip install netvis-py`). The native CLI has no Python dependency.
- **Node.js 18+ / Bun** — required only for embedding the WASM component (`npm install @netvis/wasm`) in browser or Workbench integrations.

- **Cairo or Skia** — required for PNG and PDF rasterisation; automatically linked when building from source. The pre-built binary bundles Skia.

2.1 Installing the Native CLI

The recommended installation method uses the pre-built release binary:

```
cargo install netvis
```

Alternatively, download the pre-built binary for your platform from the releases page and place it on your PATH:

```
netvis --version  
# Expected: netvis 1.0.0
```

2.2 Python Bindings

```
pip install netvis-py
```

```
import netvis  
g = netvis.load("topology.yaml")  
g.render("output.svg", layout="hierarchical")
```

2.3 WASM Component for Browser / Workbench

```
npm install @netvis/wasm  
# or: bun add @netvis/wasm
```

Deterministic Mode for CI/CD

Pass `--seed <integer>` to any render command to enable **DeterministicMode**. With a fixed seed, force-directed and other stochastic layouts produce identical output on every run, making SVG diffs in pull-request pipelines reliable. Example: `netvis -input topology.yaml -seed 42 -o output.svg`

Scale ceiling for force-directed layouts

The force-directed layout algorithm has a practical scale ceiling of approximately **10 000 nodes**. Beyond this size the iterative physics simulation becomes prohibitively slow even on modern hardware. For large-scale topologies, prefer the `multilevel` or `stress` layouts, which scale to hundreds of thousands of nodes, or use the `auto-tuner` which selects an appropriate algorithm automatically based on graph size.

3 Core Workflows

3.1 Basic Rendering

Step 1. Write your topology in YAML (see Section 5 for the full schema).

Step 2. Choose a layout algorithm appropriate for your topology type (see Section 4).

Step 3. Select an output format and resolution:

```
netvis --input topology.yaml -o output.svg
```

Use `--watch` to re-render automatically when the input file changes.

```
netvis --input topology.yaml --format png --dpi 300 -o output.png
```

```
netvis --input topology.yaml --format pdf -o diagram.pdf
```

Step 4. Apply a theme for consistent styling across documentation:

```
netvis --input topology.yaml --theme dark -o output.svg
```

3.2 Hierarchical Datacenter Visualization

The Sugiyama-based hierarchical layout is purpose-built for datacenter topologies with distinct spine, leaf, and server layers.

Step 1. Assign a layer value to each node group in your YAML (spine = 0, leaf = 1, server = 2).

Step 2. Render with the hierarchical layout:

```
netvis --input dc-topology.yaml \
  --layout hierarchical \
  --show-layers \
  -o datacenter.svg
```

Step 3. Use `--direction` to control flow direction (top-down is the default; left-right suits wide-format diagrams):

```
netvis --input dc-topology.yaml \
  --layout hierarchical \
  --direction left-right \
  --width 1600 --height 800 \
  -o datacenter-wide.svg
```

Step 4. Export to PDF for inclusion in change-management documentation:

```
netvis --input dc-topology.yaml \
  --layout hierarchical \
  --format pdf \
  --theme blueprint \
  -o datacenter.pdf
```

Grouping racks in hierarchical layouts

Add a group field to nodes to visually cluster them. NetVis draws a labelled bounding box around each group, making rack or pod boundaries immediately visible without manual annotation.

3.3 Geographic WAN Maps

For WAN topologies where nodes represent real geographic locations, the geographic layout projects nodes onto a world map.

Step 1. Annotate each node with latitude and longitude attributes in the YAML:

```
nodes:
  - id: syd
    label: "Sydney PoP"
    node_type: router
```

Coordinates must be in decimal degrees (WGS84).

```

    attributes:
      latitude: -33.8688
      longitude: 151.2093
  - id: lon
    label: "London PoP"
    node_type: router
    attributes:
      latitude: 51.5074
      longitude: -0.1278

```

Step 2. Render with the geographic layout:

```

netvis --input wan-topology.yaml \
  --layout geographic \
  --show-map \
  -o wan-map.svg

```

Step 3. Enable great-circle arcs for accurate circuit routing:

```

netvis --input wan-topology.yaml \
  --layout geographic \
  --show-map \
  --great-circle-arcs \
  --theme network \
  -o wan-map-gc.svg

```

Antimeridian wrapping

Edges that cross the 180° antimeridian (the date line) are automatically split and wrapped, but this can produce unexpected visual artefacts in some SVG viewers. If your topology spans the Pacific, verify the output carefully and consider using the `--projection cylindrical` flag to force a projection that handles wrapping cleanly.

3.4 Traffic Overlay and Animation

NetVis can overlay interface utilization data onto any topology diagram, colouring edges by traffic load.

Step 1. Prepare a traffic data file `traffic.yaml`:

```

traffic:
  timestamp: "2026-03-06T12:00:00Z"
  links:
    - from: core1
      to: core2
      utilization_pct: 78.4
    - from: core1
      to: sw1
      utilization_pct: 32.1
    - from: core2
      to: sw1
      utilization_pct: 5.3

```

Step 2. Render with the traffic overlay:

```

netvis --input topology.yaml \
  --traffic traffic.yaml \
  --traffic-colormap heat \

```

```
-o topology-traffic.svg
```

Step 3. Generate an animated SVG showing traffic over time from a directory of timestamped traffic files:

```
netvis --input topology.yaml \
  --traffic-dir traffic-snapshots/ \
  --animate \
  --animate-fps 4 \
  -o topology-animated.svg
```

Utilization colour thresholds

The default heat colormap uses green (0–60%), amber (60–85%), and red (85–100%). Override thresholds with `--traffic-thresholds 50,80` to match your network's engineering guidelines.

3.5 Topology Diffing

When reviewing planned changes or investigating incidents, the diff workflow visually highlights added, removed, and modified nodes and edges between two topology snapshots.

Step 1. Render a visual diff between two YAML snapshots:

```
netvis --input new-topology.yaml \
  --diff old-topology.yaml \
  -o topology-diff.svg
```

Added elements are shown in green, removed elements in red, and modified elements (changed label, weight, or attributes) in amber.

Step 2. Generate a text summary of changes alongside the visual output:

```
netvis --input new-topology.yaml \
  --diff old-topology.yaml \
  --diff-summary \
  -o topology-diff.svg
```

Step 3. Run in CI to fail on unexpected topology changes:

```
netvis --input new-topology.yaml \
  --diff baseline-topology.yaml \
  --diff-strict \
  -o /dev/null
# Returns exit code 1 if any differences found
```

Exit code is 1 if any differences are found; use in CI pipelines to gate changes.

3.6 Temporal Analysis

When topology snapshots are captured over time (for example, from a network management system), the `timeline` sub-command produces an annotated timeline of topology change events.

Step 1. Collect snapshots into a directory with ISO-8601 timestamps in the filenames:

```
snapshots/
  2026-03-01T00:00:00Z.yaml
  2026-03-02T00:00:00Z.yaml
  2026-03-03T00:00:00Z.yaml
```



```
2026-03-04T00:00:00Z.yaml
```

Step 2. Run the timeline analysis:

```
netvis timeline --dir snapshots/ -o timeline.svg
```

Step 3. Restrict to a specific time window:

```
netvis timeline \  
  --dir snapshots/ \  
  --from "2026-03-02T00:00:00Z" \  
  --to   "2026-03-04T00:00:00Z" \  
  -o timeline-window.svg
```

Step 4. Query for topology state at a specific point in time:

```
netvis query \  
  --dir snapshots/ \  
  --at "2026-03-03T06:00:00Z" \  
  --output topology-at-time.yaml
```

4 Layout Algorithm Guide

NetVis ships ten layout algorithms. The correct choice depends primarily on the topology type and the intended audience of the diagram.

4.1 Algorithm Summary

Table 1: NetVis layout algorithms and recommended use cases.

Algorithm	Scale	Recommended for
force-directed	≤10K nodes	Small general-purpose topologies; organic ring/mesh networks; exploratory diagrams where structure is unknown
hierarchical	≤100K nodes	Datacenter spine-leaf; protocol hierarchies; any DAG with clear tier structure
geographic	any	WAN topologies with real location data; submarine cable maps; PoP diagrams
isometric	≤50K nodes	Physical infrastructure; floor-plan DC diagrams; rack-elevation maps
radial	≤20K nodes	Hub-and-spoke; AS peer graphs; BGP route reflector topologies
stress	≤500K nodes	Large irregular topologies; graph-drawing benchmark layouts; readable cluster separation
multilevel	unlimited	Very large topologies (100K+ nodes); coarse-to-fine recursive decomposition
symmetry-aware	≤10K nodes	Network diagrams intended for documentation; symmetric ring or ladder topologies
auto-tuner	any	Unknown or mixed topologies; CI/CD pipelines where manual tuning is impractical
composite	any	Multi-domain diagrams; overlay networks; any topology where different regions suit different algorithms

4.2 Selecting an Algorithm

Pass the algorithm name to `--layout`:

```
netvis --input topology.yaml --layout stress -o output.svg
```

4.3 Composite Layouts

The composite layout allows different sub-graphs to use different algorithms. Define layout regions in the YAML using the `layout_region` node attribute:

```
nodes:
  - id: core1
    attributes:
      layout_region: "backbone"
      layout_algorithm: "stress"
  - id: dc-r1
    attributes:
      layout_region: "datacenter"
      layout_algorithm: "hierarchical"
```

```
netvis --input topology.yaml --layout composite -o output.svg
```

4.4 Auto-Tuner

The auto-tuner inspects the graph's structural properties (node count, edge density, degree distribution, detected planarity, presence of geographic coordinates) and selects and configures an algorithm automatically. It is the recommended choice for CI pipelines rendering arbitrary user-supplied topologies.

```
netvis --input topology.yaml --layout auto-tuner -o output.svg
```

Inspecting auto-tuner decisions

Pass `--layout-explain` to print a JSON summary of the auto-tuner's selection rationale, including the detected graph properties and the algorithm chosen.

5 YAML Topology Reference

5.1 Top-Level Structure

A NetVis topology file is a YAML document with a single top-level topology key:

```
topology:
  name:      "my-network"      # required: human-readable identifier
  description: "..."         # optional: freeform description
  version:   "1.0"             # optional: schema version
  nodes:     [...]            # list of node objects
  edges:     [...]            # list of edge objects
  groups:    [...]            # optional: node groupings
  layers:    [...]            # optional: layer definitions
  traffic:   [...]            # optional: inline traffic data
  annotations: [...]          # optional: diagram annotations
```

5.2 Node Fields

Table 2: Node object fields.

Field	Type	Description
id	string	Unique node identifier (required)
label	string	Display label; defaults to id if omitted
node_type	string	Semantic type: router, switch, firewall, server, cloud, unknown
attributes	map	Arbitrary key-value pairs; used by layout algorithms (e.g. latitude, longitude) and traffic overlays
position	map	Fixed position override: {x: 100, y: 200}. Nodes with a position are pinned and excluded from layout computation
group	string	Group identifier; nodes sharing a group are visually clustered
layer	integer	Layer index for hierarchical layouts; lower values appear higher in the diagram

5.3 Edge Fields

Table 3: Edge object fields.

Field	Type	Description
from	string	Source node id (required)
to	string	Target node id (required)
label	string	Edge label displayed at midpoint
weight	float	Edge weight; used by force-directed spring constant and by stress layout
directed	bool	Render as a directed (arrowed) edge; default false
attributes	map	Arbitrary key-value pairs; used by traffic overlay matching

5.4 Groups

Groups define named clusters rendered as labelled bounding boxes:

```
groups:
- id: spine-pod-1
  label: "Spine Pod 1"
  style: dashed           # solid | dashed | dotted
  color: "#3b82f6"       # optional fill/border colour
  nodes: [core1, core2]
```

5.5 Layers

Layer definitions control vertical ordering in hierarchical and isometric layouts:

```
layers:
- id: 0
  label: "Spine"
  color: "#eff6ff"
- id: 1
  label: "Leaf"
  color: "#f0fdf4"
- id: 2
  label: "Server"
  color: "#fefce8"
```

5.6 Inline Traffic Data

Traffic data can be embedded directly in the topology file or supplied separately via `--traffic`:

```
traffic:
- from: core1
  to: core2
  utilization_pct: 72.0
  bps: 7200000000
```

5.7 Annotations

Annotations add free-form text or shape overlays to the rendered diagram:

```
annotations:
- type: text
  text: "Primary path"
  x: 400
  y: 100
  style: bold
- type: arrow
```

```

from_xy: [200, 300]
to_xy:   [400, 300]
label: "Failover direction"

```

6 CLI Command Reference

6.1 Core Commands

Core Sub-commands

Command	Description
netvis render	Render a topology to SVG/PNG/PDF (default sub-command)
netvis diff	Visually diff two topology YAML files
netvis timeline	Analyse topology changes across a snapshot directory
netvis query	Query topology state from a snapshot archive
netvis validate	Validate a topology YAML file against the schema
netvis version	Print version and build information

6.2 Layout Flags

Layout Options

Command	Description
--layout <algo>	Layout algorithm (see Table 1)
--direction <dir>	Flow direction: top-down, bottom-up, left-right, right-left
--seed <n>	Fixed random seed for deterministic layouts
--iterations <n>	Force-directed iteration count (default: 500)
--layout-explain	Print JSON layout selection rationale to stderr
--show-layers	Draw layer bands behind nodes
--projection <name>	Map projection for geographic layout: mercator, cylindrical, equirectangular

6.3 Appearance Flags

Appearance Options

Command	Description
--theme <name>	Visual theme (see Section 7)
--width <px>	Canvas width in pixels (default: 1200)
--height <px>	Canvas height in pixels (default: 800)
--dpi <n>	DPI for PNG/PDF output (default: 96)
--font-size <pt>	Base label font size (default: 12)
--edge-width <px>	Default edge stroke width (default: 1.5)
--node-radius <px>	Default node radius (default: 18)
--lod <mode>	Level-of-detail mode: auto, none, conservative, balanced, aggressive
--show-map	Render world-map underlay (geographic layout)
--great-circle-arcs	Route WAN edges as great-circle arcs

6.4 Filtering Flags

Filtering Options

Command	Description
--filter-group <id>	Render only nodes in the specified group
--filter-layer <n>	Render only nodes in the specified layer
--filter-type <type>	Render only nodes of the specified node_type
--exclude-group <id>	Exclude a named group from the rendered output
--highlight-node <id>	Highlight a specific node and its neighbours
--depth <n>	When used with --highlight-node, include neighbours up to <i>n</i> hops away

6.5 Traffic Overlay Flags

Traffic Overlay Options

Command	Description
--traffic <file>	Load utilization data from a YAML traffic file
--traffic-dir <dir>	Load time-series traffic from a directory
--traffic-colormap <m>	Colormap: heat, cool, traffic-light, greens
--traffic-thresholds	Override utilization thresholds (comma-separated percentages)
--animate	Produce an animated SVG from time-series traffic
--animate-fps <n>	Animation frame rate (default: 2)

6.6 Diff and Analysis Flags

Diff and Analysis Options

Command	Description
--diff <file>	Compute visual diff against a baseline YAML
--diff-summary	Emit a change-summary alongside the diff diagram
--diff-strict	Return exit code 1 if any differences are found
netvis timeline --dir	Snapshot directory for temporal analysis
--from <iso8601>	Start of timeline window
--to <iso8601>	End of timeline window
netvis query --at	Point-in-time for snapshot interpolation
--output <file>	Write reconstructed topology YAML to file

6.7 Advanced Flags

Advanced Options

Command	Description
--watch	Re-render when the input file changes
--parallel <n>	Number of render threads (default: CPU count)
--no-cache	Disable layout cache for force-directed layouts
--debug-layout	Overlay layout debug information on render
--emit-graphml	Also emit a GraphML export of the topology
--strict	Treat YAML warnings as errors

7 Themes and Level of Detail

7.1 Available Themes

NetVis ships seven built-in themes. A theme controls background colour, node palette, edge colours, font choices, and shadow/filter effects.

Table 4: Built-in NetVis themes.

Theme	Background	Best for
network	White	General-purpose network diagrams; documentation
dark	Dark	Dashboard embedding; NOC wall displays
light	Off-white	Print-friendly; documentation with light images
blueprint	Dark blue	Engineering design reviews; blueprint-style docs
solarized-dark	Solarized	Terminals; developer tooling integrations
dracula	Dark grey	Developer presentations; terminal screenshots
tokyo-night	Deep navy	Modern presentation slides; dark-mode docs

```
netvis --input topology.yaml --theme blueprint -o output.svg
```

7.2 Level of Detail (LoD)

When rendering large topologies, label and icon rendering can be selectively suppressed to reduce clutter and SVG file size. The --lod flag controls this behaviour:

Table 5: Level-of-detail modes.

Mode	Behaviour
auto	NetVis selects a mode based on node count: none below 200, conservative 200–2 000, balanced 2 000–20 000, aggressive above 20 000
none	All labels, icons, and edge annotations rendered
conservative	Suppress edge weight labels; reduce icon size
balanced	Suppress all edge labels; hide low-degree node labels
aggressive	Render nodes as dots; suppress all labels and icons

LoD and SVG filter performance

SVG drop-shadow and glow filters are disabled automatically when `--lod` balanced or aggressive is active, because browser rendering of SVG filters degrades significantly at 50+ filtered elements. If you require shadow effects on large diagrams, use PNG output where Skia applies filters during rasterisation.

7.3 Custom Themes

Define a custom theme in YAML and reference it by file path:

```
# mytheme.yaml
theme:
  background: "#1e1e2e"
  node_fill:   "#313244"
  node_stroke: "#89b4fa"
  edge_color:  "#6c7086"
  label_color: "#cdd6f4"
  font_family: "JetBrains Mono, monospace"
  shadow: false
```

```
netvis --input topology.yaml --theme mytheme.yaml -o output.svg
```

8 Troubleshooting

Problem 1: Force-directed layout is extremely slow or hangs on topologies larger than 10 000 nodes

Cause: The iterative physics simulation has $O(n^2)$ per-iteration complexity for the repulsion calculation. Beyond the 10 000-node scale ceiling the simulation may take hours or fail to converge.

Solution: Switch to the multilevel or stress layout, which use hierarchical coarsening and are efficient at 100 000+ nodes: `--layout multilevel`. Alternatively use `--layout auto-tuner` to let NetVis select an appropriate algorithm based on graph size automatically.

Problem 2: Node labels overlap heavily in large or dense topologies

Cause: The default greedy label placement algorithm makes a single left-to-right pass and does not backtrack to resolve overlaps caused by later labels. At high node densities the result can be illegible.

Solution: Enable the force-repulsion label placer with `--label-placement force`, which iteratively nudges labels to reduce overlap at the cost of higher render time. Alternatively, use `--lod balanced` to suppress labels on low-degree nodes, or increase canvas size with `--width` and `--height`.

Problem 3: SVG output differs between runs on the same input file

Cause: The force-directed and symmetry-aware layout algorithms use a pseudo-random number generator seeded from the system clock when `--seed` is not specified. Without `DeterministicMode` the initial node placement differs on every run, leading to different final layouts.

Solution: Pass a fixed seed: `--seed 42`. All stochastic layout algorithms use the seed when provided, producing identical SVG output across runs. Commit the seed value alongside your topology file for reproducible CI diagrams.

Problem 4: WAN edges crossing the 180° antimeridian appear as horizontal lines spanning the full map width

Cause: The default Mercator projection clips edge paths at the antimeridian without wrapping them, producing an incorrect horizontal artefact.

Solution: Use `--projection cylindrical`, which handles antimeridian wrapping correctly. Alternatively, if your topology spans the Pacific, split the edge into two segments meeting at a virtual PoP near the antimeridian.

Problem 5: SVG diagrams load slowly or cause browser tab freezing at 50+ nodes with drop shadows

Cause: SVG filter primitives (`feDropShadow`, `feGaussianBlur`) are applied per-element by the browser's rendering engine. At 50+ filtered elements many browsers repaint the entire filter region on every scroll or zoom event.

Solution: Use PNG output for large diagrams where filters are needed: `--format png --dpi 150`. Skia applies filters during rasterisation, so the delivered PNG has no per-element filter overhead. For SVG output, use `--lod balanced` which disables SVG filters automatically above the threshold, or use `--no-shadows` to disable shadows globally while retaining other SVG features.

9 Integration with Other Tools

NetVis is designed to fit into a broader network automation ecosystem. The following integrations are supported natively or via adapters.

9.1 TopoGen — Topology Generation

TopoGen generates seed-deterministic synthetic topologies for testing and benchmarking. Pipe TopoGen output directly into NetVis to visualize generated topologies without writing intermediate files:

```
topogen generate \
  --template fat-tree --k 4 \
  --seed 42 \
  --format netvis \
  | netvis --stdin --layout hierarchical -o fat-tree.svg
```

See also: *TopoGen User Manual* (TOPOGEN-UM-001) — Topology templates, seed parameters, and the NetVis output format.

9.2 netsim — Simulation Results Overlay

netsim produces per-link utilization and latency data as part of discrete-event simulations. Export simulation results in the NetVis traffic format and overlay them on the topology:

```
netsim run simulation.yaml \
  --export-traffic traffic-sim.yaml \
  --format netvis
netvis --input topology.yaml \
  --traffic traffic-sim.yaml \
  --traffic-colormap heat \
  -o sim-overlay.svg
```

See also: *netsim User Manual* (NETSIM-UM-001) — Traffic export formats and simulation result interpretation.

9.3 AutoNetKit — Topology Modelling

AutoNetKit (ANK) builds abstract topology models and validates them against type constraints. Export an ANK topology to NetVis format for diagram generation:

```
ank build topology.yaml \
  --export-format netvis \
  --output ank-export.yaml
netvis --input ank-export.yaml \
  --layout hierarchical \
  --theme network \
  -o ank-topology.svg
```

See also: *AutoNetKit User Manual* (ANK-UM-001) — Export formats and type-safe topology modelling.

9.4 NetBox — IPAM/DCIM Adapter

The netvis-netbox adapter pulls device and circuit data from a NetBox instance and converts it to NetVis YAML topology format:

```
netvis-netbox pull \
  --url https://netbox.example.com \
  --token "$NETBOX_TOKEN" \
  --site SYD-DC1 \
  --output topology-netbox.yaml
netvis --input topology-netbox.yaml \
  --layout hierarchical \
  -o netbox-diagram.svg
```

NetBox adapter filtering

Use `--role` and `--tag` flags on the adapter to restrict the exported topology to specific device roles or NetBox tags, avoiding over-large diagrams from full-site exports.

See also: *NetBox Adapter Reference* — Connection, filtering, and incremental sync options.

9.5 Workbench — Embedded WASM Component

The NetVis WASM component is embedded in the Workbench interactive notebook environment. Topology cells in Workbench render live, interactive NetVis diagrams directly in the browser:

```
# Workbench topology cell header
%% netvis
layout: hierarchical
theme: dark
interactive: true
---
topology:
  name: workbench-demo
  nodes: [...]
  edges: [...]
```

Pan, zoom, and node-drag interactions are available in the Workbench WASM component. Click a node to open its attribute inspector panel.

See also: *Workbench User Manual* (WKBENCH-UM-001) — Topology cell syntax, interactive features, and exporting diagrams from notebooks.

10 Further Reading

- *NetVis Technical Report* (NETVIS-TR-001) — Architecture, layout algorithm design rationale, performance benchmarks, and the Rust rendering pipeline internals.
- *TopoGen User Manual* (TOPOGEN-UM-001) — Generating synthetic topologies for testing NetVis at scale.
- *AutoNetKit User Manual* (ANK-UM-001) — Type-safe topology modelling upstream of NetVis visualization.
- *Workbench User Manual* (WKBENCH-UM-001) — Embedding NetVis as an interactive WASM component in network automation notebooks.

A Topology YAML Schema

Listing 1: Full YAML topology schema (abbreviated)

```
name: string           # Required: topology name
render:                # Optional: rendering hints
  preset: string        # Layout preset (default|sugiyama|radial|geo)
  theme: string          # Theme (network|dark|light)
  width: number          # Canvas width in pixels
  height: number         # Canvas height in pixels
  lod: string            # LOD profile (none|conservative|balanced|aggressive|auto)
nodes:                 # Required: list of nodes
  - id: string           # Unique node identifier
    label: string        # Display label (optional, defaults to id)
    node_type: string     # Device type (router|switch|server|firewall|cloud)
    attributes: map       # Arbitrary key-value metadata
    position:            # Position hint (optional)
      x: number
      y: number
    group: string         # Group membership (optional)
    layer: string         # Layer membership (optional)
edges:                 # Required: list of edges
  - from: string         # Source node ID
    to: string            # Target node ID
    id: string            # Explicit edge ID (optional, for traffic/filter targeting)
```

```

    label: string          # Edge label (optional)
    weight: number         # Edge weight (default: 1.0)
    directed: boolean      # Directed edge (default: false)
    attributes: map        # Arbitrary key-value metadata
  groups:                 # Optional: group definitions
    - id: string
      label: string
      style: map           # Boundary style overrides
  layers:                 # Optional: layer definitions for isometric layout
    - id: string
      label: string
      order: number        # Stacking order (lower = bottom)
  traffic:                 # Optional: traffic flow animation overlay
  edges:
    - id: string           # Edge ID (matches edges[].id)
      utilization: number  # 0.0--1.0 utilization level
      direction: string    # forward/reverse/bidirectional
      style: string        # dot/dash animation style
  annotations:            # Optional: SVG annotation markup
    - anchor: string       # Node ID to anchor near
      type: string         # text/callout
      text: string         # Annotation text
      shape: string        # circle/box/arrow (callout only)
      layer: string        # Annotation layer name

```

B Configuration Reference

B.1 CLI Flags

B.1.1 Input and Output

Flag	Type	Default	Description
--input, -i	path	(required)	Topology file; globs and - for stdin
--output, -o	path	auto	Output file path
--config	path	—	JSON config file
--format	string	svg	Output format: svg, png, pdf, html
--dpi	integer	96	DPI for PNG export

B.1.2 Layout

Flag	Default	Description
--layout	auto	Algorithm: force-directed, hierarchical, radial, isometric, geographic
--preset	—	Layout preset: default, wan, datacenter, sugiyama, geographic, ...
--template	—	Topology template: spine-leaf, hub-spoke, ring, multi-tier
--seed	42	RNG seed for deterministic layout

B.1.3 Appearance

Flag	Default	Description
--theme	network	Theme (16 built-in): light, dark, network, blueprint, datacenter, print, high-contrast, solarized-{dark,light}, nord, catppuccin, brewer, dracula, tokyo-night, monokai, cyberpunk
--labels	label	Label mode: off, label, id
--node-size	auto	Node radius in scene units
--lod	auto	LOD profile: none, conservative, balanced, aggressive, auto

B.1.4 Canvas and Viewport

Flag	Default	Description
--width	1200	SVG viewport width (px)
--height	800	SVG viewport height (px)
--padding	40	ViewBox padding (px)
--padding-ratio	0.06	Padding as fraction of scene

B.1.5 Force-Directed Tuning

Flag	Default	Description
--link-distance	30.0	Target link length (scene units)
--link-strength	auto	Edge spring strength (0.0–1.0)
--charge-strength	–30.0	Many-body charge (negative = repel)
--max-iterations	300	Maximum simulation iterations
--soft-anchor-strength	0.0	Anchor strength for hinted nodes
--refine	false	Apply post-layout edge refinement

B.1.6 Geographic Layout

Flag	Default	Description
--show-map	false	Show background coastline map
--great-circle-arcs	false	Render edges as curved arcs
--show-distances	false	Show geodesic distance labels
--distance-unit	km	Distance unit: km or mi

B.1.7 Filtering

Flag	Description
--show-type	Only render nodes of these types (comma-separated, repeatable)
--hide-type	Hide nodes of these types
--show-tag	Only render nodes with these tags
--hide-tag	Hide nodes with any of these tags
--show-group	Only render nodes in these groups
--hide-group	Hide nodes in these groups
--filter-file	Load filter rules from YAML file
--path	BFS path query between two node IDs
--highlight-path	Highlight a named path (repeatable)

B.1.8 Analysis Features

Flag	Description
--diff <path>	Compare against a base topology and visualize changes
--traffic <path>	Traffic overlay configuration YAML
--annotations <path>	Annotation markup YAML
--hide-layers	Suppress named annotation layers
--show-layers	Force-show named annotation layers
--cluster	Enable automatic community detection
--algorithm	Clustering algorithm: louvain, label-propagation, edge-betweenness
--metric <field>	Map a node attribute to color

B.1.9 Quality and Diagnostics

Flag	Default	Description
--quality-threshold	85.0	Quality score warning threshold
--fail-on-low-quality	false	Exit with error if below threshold
--deterministic	true	Deterministic output mode
--diagnostics	—	Structured diagnostics: json, yaml, ndjson
--verbose, -v	false	Show debug statistics

B.2 CLI Subcommands

Command	Description
list-examples	List built-in topology examples
print-example <name>	Print a built-in topology to stdout
list-templates	List layout templates
list-adapters	List data source adapters
import	Import topology from adapter into YAML
schema	Print JSON Schema (config, topology, or diagnostics)
print-config <name>	Print an example config to stdout
validate	Validate topology/config against schema
migrate	Normalize files to current schema version
check-quality	Analyze SVG files for quality issues
diff	Compare two topologies, visualize changes
timeline	Time-series topology evolution analysis
query	Topology query operations

B.3 JSON Configuration File

The `--config` flag accepts a JSON file with the following structure:

Listing 2: NetVisConfig JSON

```
{
  "schema_version": "1.0",
  "layout": "force_directed",
  "quality": "balanced",
  "width": 1200.0,
  "height": 800.0,
  "effects": {
    "max_filtered_elements": 50
  }
}
```

Key	Type	Default	Description
layout	string	force_directed	Layout algorithm
quality	string	balanced	Quality preset: quick, balanced, quality
width	float	800.0	Viewport width
height	float	600.0	Viewport height
effects.max_filtered_elements	int	50	SVG filter budget

Table 6: JSON configuration keys.

Quality presets control iteration multipliers: quick (0.33x), balanced (1.0x), quality (3.0x).